



Hey! Check out my new Git course on [GitByBit.com](https://gitbybit.com)!

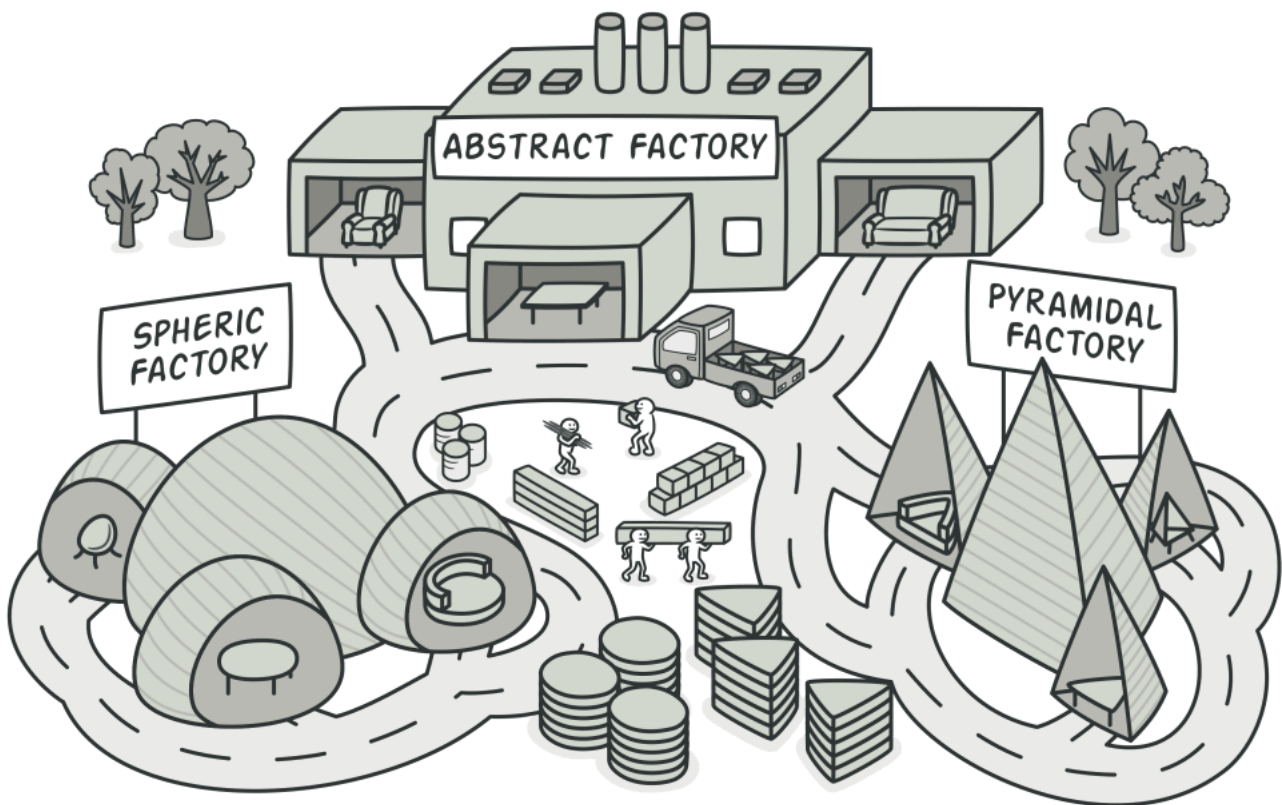


[Home](#) / [Design Patterns](#) / [Creational Patterns](#)

Abstract Factory

Intent

Abstract Factory is a creational design pattern that lets you produce families of related objects without specifying their concrete classes.



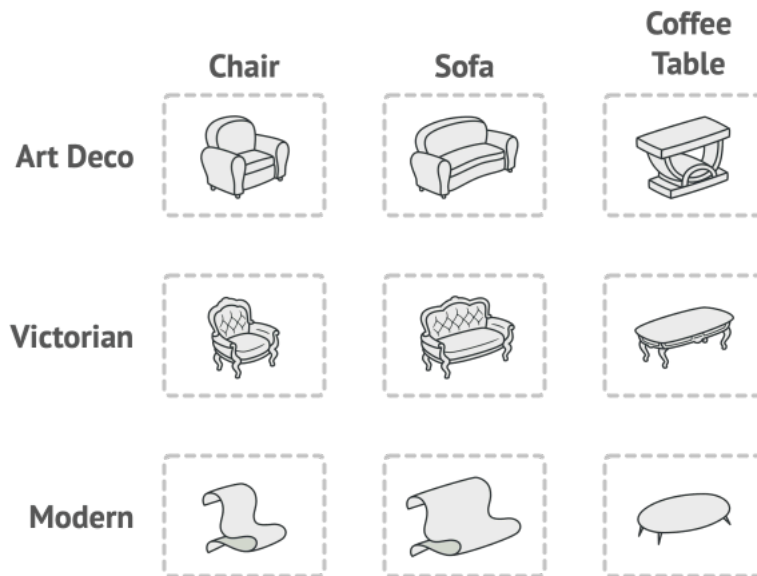
Problem

Imagine that you're creating a furniture shop simulator. Your code consists of classes that represent:

1. A family of related products, say: `Chair` + `Sofa` + `CoffeeTable`.

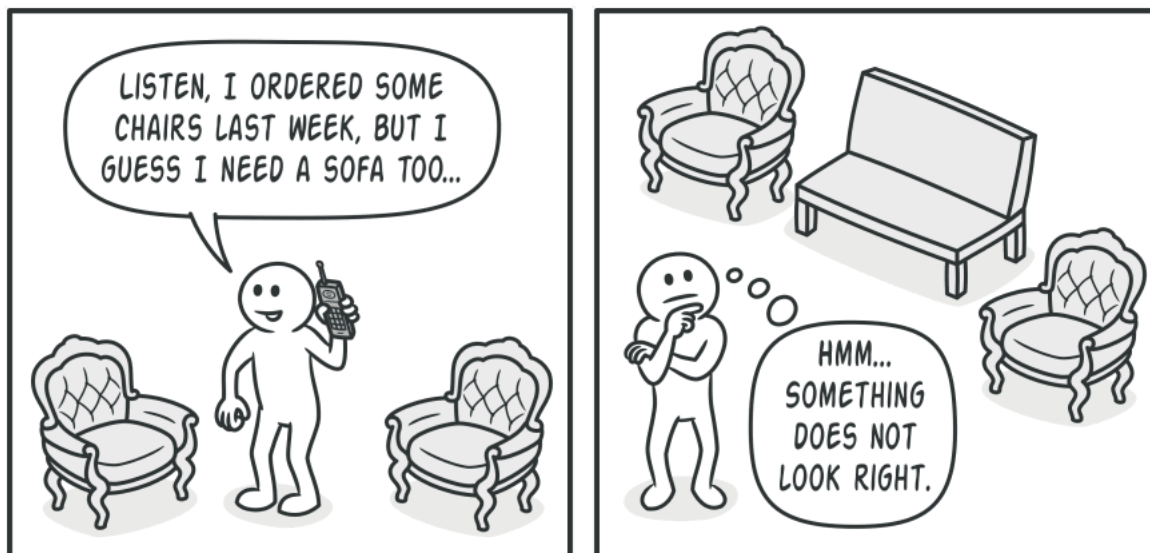


Hey! Check out my new Git course on [GitByBit.com](https://gitbybit.com)!



Product families and their variants.

You need a way to create individual furniture objects so that they match other objects of the same family. Customers get quite mad when they receive non-matching furniture.



A Modern-style sofa doesn't match Victorian-style chairs.

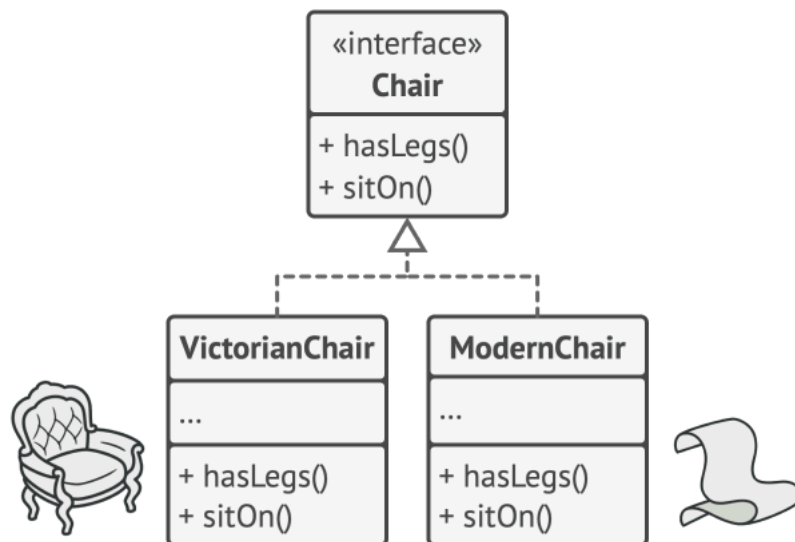
Also, you don't want to change existing code when adding new products or families of products to the program. Furniture vendors update their catalogs very often, and you wouldn't want to change the core code each time it happens.



Hey! Check out my new Git course on [GitByBit.com](https://gitbybit.com)!

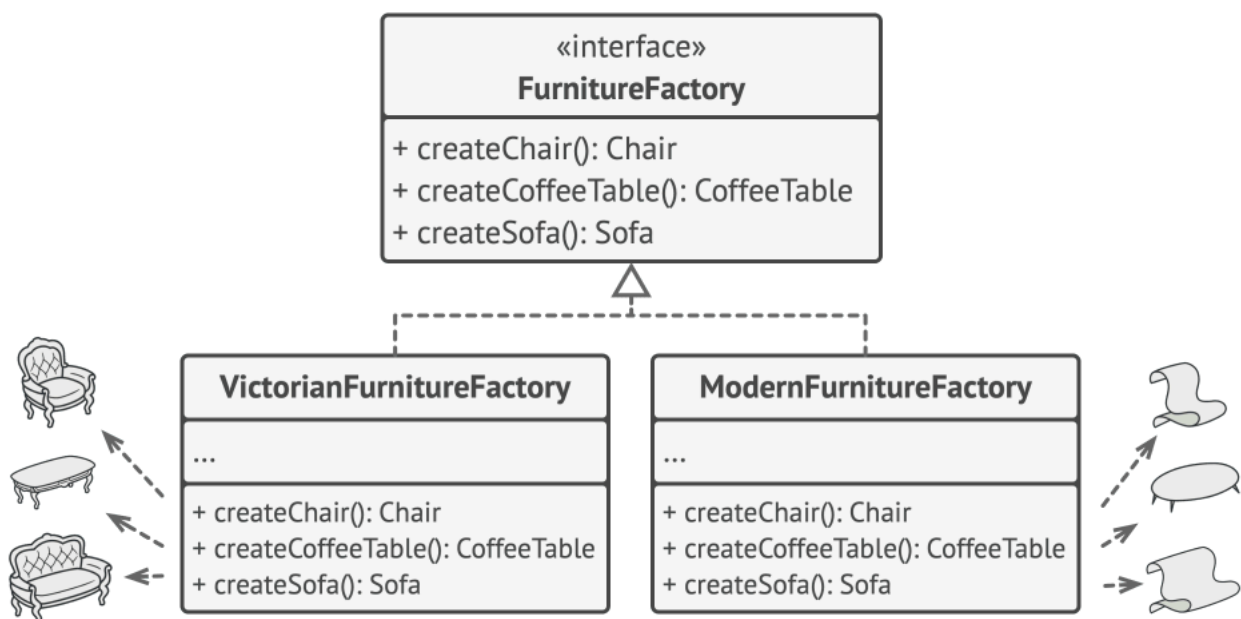


The first thing the Abstract Factory pattern suggests is to explicitly declare interfaces for each distinct product of the product family (e.g., chair, sofa or coffee table). Then you can make all variants of products follow those interfaces. For example, all chair variants can implement the `Chair` interface; all coffee table variants can implement the `CoffeeTable` interface, and so on.



All variants of the same object must be moved to a single class hierarchy.

The next move is to declare the *Abstract Factory*—an interface with a list of creation methods for all products that are part of the product family (for example, `createChair`, `createSofa` and `createCoffeeTable`). These methods must return **abstract** product types represented by the interfaces we extracted previously: `Chair`, `Sofa`, `CoffeeTable` and so on.



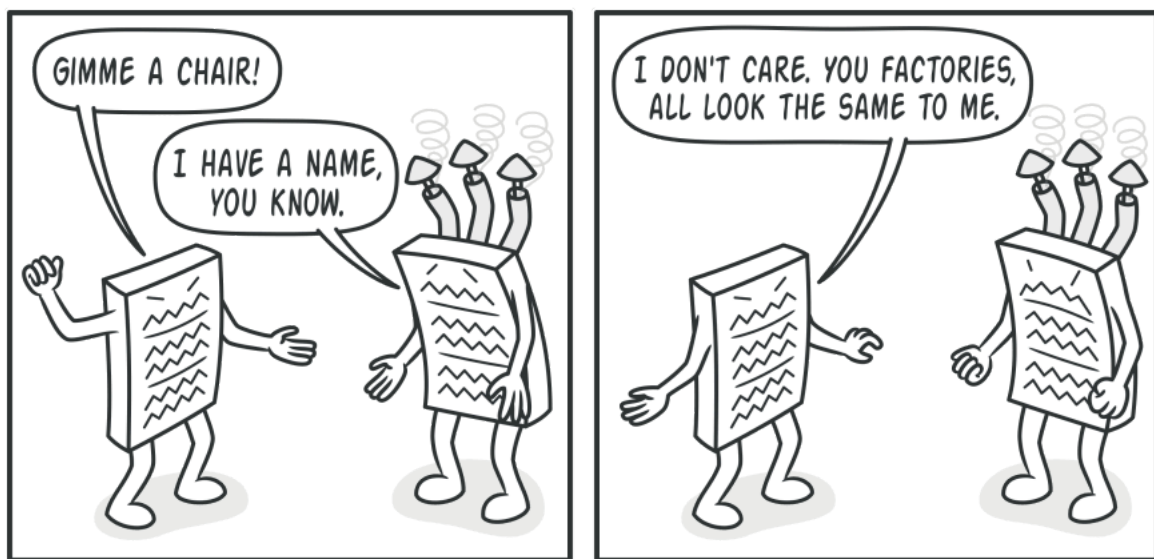


Hey! Check out my new Git course on [GitByBit.com](https://gitbybit.com)!



Now, how about the product variants? For each variant of a product family, we create a separate factory class based on the `AbstractFactory` interface. A factory is a class that returns products of a particular kind. For example, the `ModernFurnitureFactory` can only create `ModernChair`, `ModernSofa` and `ModernCoffeeTable` objects.

The client code has to work with both factories and products via their respective abstract interfaces. This lets you change the type of a factory that you pass to the client code, as well as the product variant that the client code receives, without breaking the actual client code.

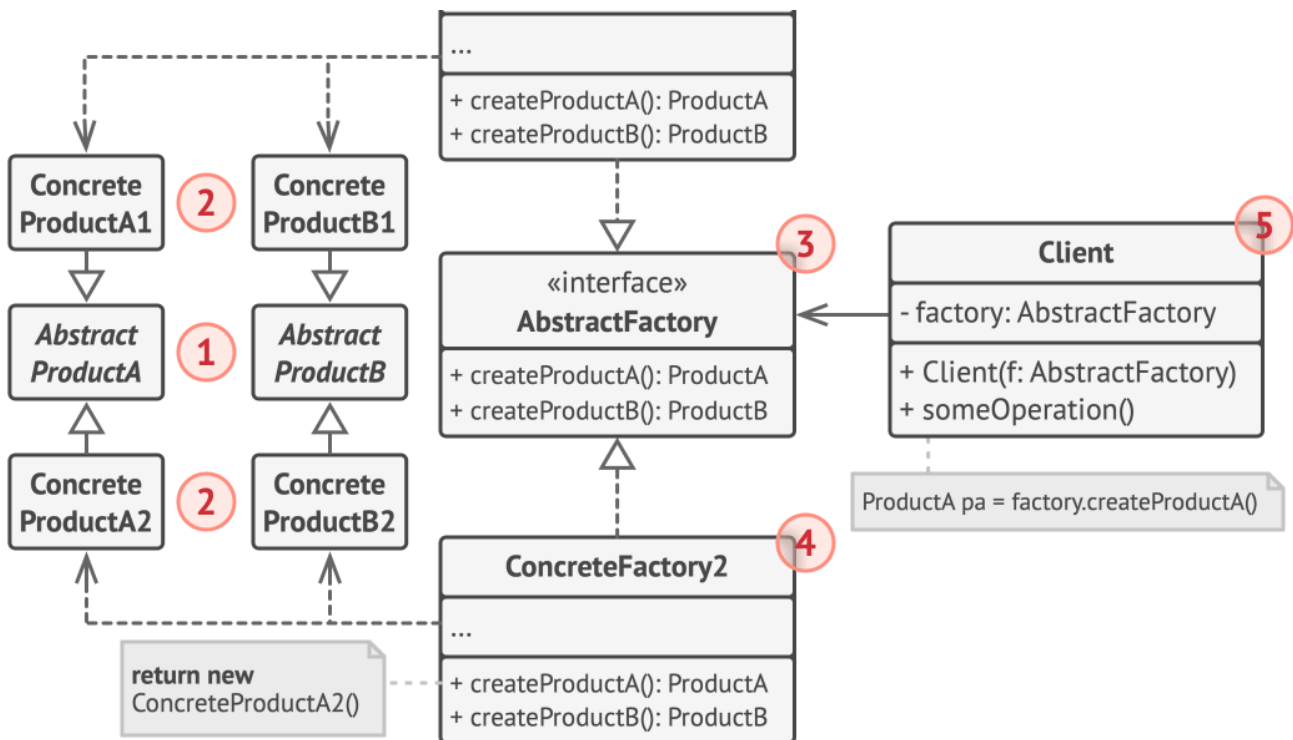


The client shouldn't care about the concrete class of the factory it works with.

Say the client wants a factory to produce a chair. The client doesn't have to be aware of the factory's class, nor does it matter what kind of chair it gets. Whether it's a Modern model or a Victorian-style chair, the client must treat all chairs in the same manner, using the abstract `Chair` interface. With this approach, the only thing that the client knows about the chair is that it implements the `sitOn` method in some way. Also, whichever variant of the chair is returned, it'll always match the type of sofa or coffee table produced by the same factory object.

There's one more thing left to clarify: if the client is only exposed to the abstract interfaces, what creates the actual factory objects? Usually, the application creates a concrete factory object at the initialization stage. Just before that, the app must select the factory type depending on the configuration or the environment settings.

Structure

Hey! Check out my new Git course on [GitByBit.com](https://gitbybit.com)!

1. **Abstract Products** declare interfaces for a set of distinct but related products which make up a product family.
2. **Concrete Products** are various implementations of abstract products, grouped by variants. Each abstract product (chair/sofa) must be implemented in all given variants (Victorian/Modern).
3. The **Abstract Factory** interface declares a set of methods for creating each of the abstract products.
4. **Concrete Factories** implement creation methods of the abstract factory. Each concrete factory corresponds to a specific variant of products and creates only those product variants.
5. Although concrete factories instantiate concrete products, signatures of their creation methods must return corresponding *abstract* products. This way the client code that uses a factory doesn't get coupled to the specific variant of the product it gets from a factory. The **Client** can work with any concrete factory/product variant, as long as it communicates with their objects via abstract interfaces.

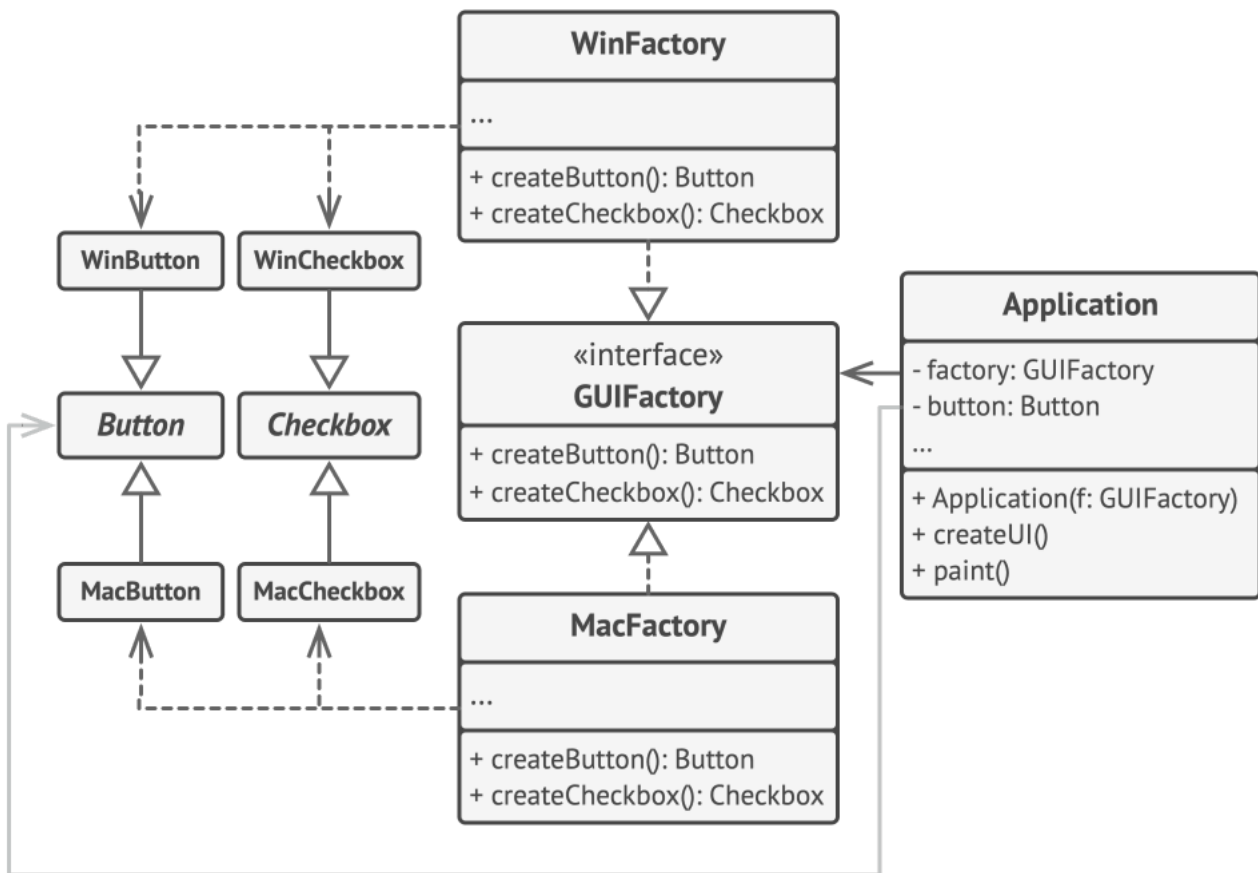
Pseudocode



Hey! Check out my new Git course on [GitByBit.com](https://gitbybit.com)!



created elements consistent with a selected operating system.



The cross-platform UI classes example.

The same UI elements in a cross-platform application are expected to behave similarly, but look a little bit different under different operating systems. Moreover, it's your job to make sure that the UI elements match the style of the current operating system. You wouldn't want your program to render macOS controls when it's executed in Windows.

The Abstract Factory interface declares a set of creation methods that the client code can use to produce different types of UI elements. Concrete factories correspond to specific operating systems and create the UI elements that match that particular OS.

It works like this: when an application launches, it checks the type of the current operating system. The app uses this information to create a factory object from a class that matches the operating system. The rest of the code uses this factory to create UI elements. This prevents the wrong elements from being created.

With this approach, the client code doesn't depend on concrete classes of factories and UI elements as long as it works with these objects via their abstract interfaces. This also lets the



Hey! Check out my new Git course on [GitByBit.com](https://gitbybit.com)!



As a result, you don't need to modify the client code each time you add a new variation of UI elements to your app. You just have to create a new factory class that produces these elements and slightly modify the app's initialization code so it selects that class when appropriate.

```
// The abstract factory interface declares a set of methods that
// return different abstract products. These products are called
// a family and are related by a high-level theme or concept.
// Products of one family are usually able to collaborate among
// themselves. A family of products may have several variants,
// but the products of one variant are incompatible with the
// products of another variant.
```

```
interface GUIFactory is
```

```
    method createButton():Button
    method createCheckbox():Checkbox
```

```
// Concrete factories produce a family of products that belong
// to a single variant. The factory guarantees that the
// resulting products are compatible. Signatures of the concrete
// factory's methods return an abstract product, while inside
// the method a concrete product is instantiated.
```

```
class WinFactory implements GUIFactory is
```

```
    method createButton():Button is
        return new WinButton()
    method createCheckbox():Checkbox is
        return new WinCheckbox()
```

```
// Each concrete factory has a corresponding product variant.
```

```
class MacFactory implements GUIFactory is
```

```
    method createButton():Button is
        return new MacButton()
    method createCheckbox():Checkbox is
        return new MacCheckbox()
```

```
// Each distinct product of a product family should have a base
// interface. All variants of the product must implement this
// interface.
```

```
interface Button is
```

```
    method paint()
```

```
// Concrete products are created by corresponding concrete
// factories.
```

```
class WinButton implements Button is
```

```
    method paint() is
        // Render a button in Windows style.
```

```
class MacButton implements Button is
```



Hey! Check out my new Git course on [GitByBit.com](https://gitbybit.com)!



```
// Here's the base interface of another product. All products
// can interact with each other, but proper interaction is
// possible only between products of the same concrete variant.
interface Checkbox is
    method paint()

class WinCheckbox implements Checkbox is
    method paint() is
        // Render a checkbox in Windows style.

class MacCheckbox implements Checkbox is
    method paint() is
        // Render a checkbox in macOS style.

// The client code works with factories and products only
// through abstract types: GUIFactory, Button and Checkbox. This
// lets you pass any factory or product subclass to the client
// code without breaking it.
class Application is
    private field factory: GUIFactory
    private field button: Button
    constructor Application(factory: GUIFactory) is
        this.factory = factory
    method createUI() is
        this.button = factory.createButton()
    method paint() is
        button.paint()

// The application picks the factory type depending on the
// current configuration or environment settings and creates it
// at runtime (usually at the initialization stage).
class ApplicationConfigurator is
    method main() is
        config = readApplicationConfigFile()

        if (config.OS == "Windows") then
            factory = new WinFactory()
        else if (config.OS == "Mac") then
            factory = new MacFactory()
        else
            throw new Exception("Error! Unknown operating system.")

        Application app = new Application(factory)
```



Hey! Check out my new Git course on [GitByBit.com](https://gitbybit.com)!



- 🔧 Use the Abstract Factory when your code needs to work with various families of related products, but you don't want it to depend on the concrete classes of those products—they might be unknown beforehand or you simply want to allow for future extensibility.
 - ⚡ The Abstract Factory provides you with an interface for creating objects from each class of the product family. As long as your code creates objects via this interface, you don't have to worry about creating the wrong variant of a product which doesn't match the products already created by your app.
-
- 🔧 Consider implementing the Abstract Factory when you have a class with a set of Factory Methods that blur its primary responsibility.
 - ⚡ In a well-designed program *each class is responsible only for one thing*. When a class deals with multiple product types, it may be worth extracting its factory methods into a stand-alone factory class or a full-blown Abstract Factory implementation.

📋 How to Implement

1. Map out a matrix of distinct product types versus variants of these products.
2. Declare abstract product interfaces for all product types. Then make all concrete product classes implement these interfaces.
3. Declare the abstract factory interface with a set of creation methods for all abstract products.
4. Implement a set of concrete factory classes, one for each product variant.
5. Create factory initialization code somewhere in the app. It should instantiate one of the concrete factory classes, depending on the application configuration or the current environment. Pass this factory object to all classes that construct products.
6. Scan through the code and find all direct calls to product constructors. Replace them with calls to the appropriate creation method on the factory object.

⚖️ Pros and Cons



Hey! Check out my new Git course on [GitByBit.com](https://gitbybit.com)!



with each other.

interfaces and classes are introduced along with the pattern.

- ✓ You avoid tight coupling between concrete products and client code.
- ✓ *Single Responsibility Principle*. You can extract the product creation code into one place, making the code easier to support.
- ✓ *Open/Closed Principle*. You can introduce new variants of products without breaking existing client code.

↔ Relations with Other Patterns

- Many designs start by using **Factory Method** (less complicated and more customizable via subclasses) and evolve toward **Abstract Factory**, **Prototype**, or **Builder** (more flexible, but more complicated).
- **Builder** focuses on constructing complex objects step by step. **Abstract Factory** specializes in creating families of related objects. *Abstract Factory* returns the product immediately, whereas *Builder* lets you run some additional construction steps before fetching the product.
- **Abstract Factory** classes are often based on a set of **Factory Methods**, but you can also use **Prototype** to compose the methods on these classes.
- **Abstract Factory** can serve as an alternative to **Facade** when you only want to hide the way the subsystem objects are created from the client code.
- You can use **Abstract Factory** along with **Bridge**. This pairing is useful when some abstractions defined by *Bridge* can only work with specific implementations. In this case, *Abstract Factory* can encapsulate these relations and hide the complexity from the client code.
- **Abstract Factories**, **Builders** and **Prototypes** can all be implemented as **Singletons**.

</> Code Examples



Hey! Check out my new Git course on [GitByBit.com](https://gitbybit.com)!



Extra Content

- Read our [Factory Comparison](#) to learn more about the differences between various factory patterns and concepts.



Support our free website and own the eBook!

- 22 design patterns and 8 principles explained in depth.
- 409 well-structured, easy to read, jargon-free pages.
- 225 clear and helpful illustrations and diagrams.
- An archive with code examples in 11 languages.
- All devices supported: PDF/EPUB/MOBI/KFX formats.

 [Learn more...](#)



Hey! Check out my new Git course on [GitByBit.com](https://gitbybit.com)!



← Factory Method

Factory Comparison →

[Home](#) [Refactoring](#) [Design Patterns](#) [Premium Content](#)
[Git Course](#) [Forum](#) [Contact us](#)



© 2014-2026 Refactoring.Guru. All rights reserved.

Illustrations by Dmitry Zhart

[Terms & Conditions](#) [Privacy Policy](#)

[Content Usage Policy](#) [About us](#)

Ukrainian office:

FOP Olga Skobeleva

Abolmasova 7

Kyiv, Ukraine, 02002

Email:

support@refactoring.guru

Spanish office:

Oleksandr Shvets

Avda Pamplona 64

Pamplona, Spain, 31009

Email:

support@refactoring.guru